Contenido

Monitores 1
Condición de Carrera 3
Deadlock 4
Livelock 6
Mutex (Mutua exclusión) 8
Happens-before 9
Bloqueo Intrínseco 11
Starvation 13
Thread-safe 15
Semaphore 17
Liveness 18

Monitores

En Java, los monitores se implementan mediante el uso de la palabra clave synchronized**, que permite que solo un hilo acceda a un recurso compartido a la vez.** Esto asegura exclusión mutua y evita condiciones de carrera en programas concurrentes. Cada objeto en Java puede actuar como un monitor.

Concepto de Monitores en Java

- **Definición**: Un monitor es un mecanismo de sincronización que controla el acceso de múltiples hilos a recursos compartidos.
- **Objetivo**: Garantizar que solo un hilo ejecute un bloque de código crítico a la vez.
- **Implementación en Java**:
    - Cada objeto tiene un monitor implícito.
    - Se accede mediante synchronized en métodos o bloques de código.
    - Solo un hilo puede poseer el bloqueo del monitor en un momento dado.

```
class Contador {
    private int valor = 0;

    // Método sincronizado: solo un hilo puede ejecutarlo a la vez
    public synchronized void incrementar() {
        valor++;
    }
```

```
    public synchronized int obtenerValor() {
        return valor;
    }
}

// Clase que representa un hilo que incrementa el contador
class HiloIncrementador extends Thread {
    private Contador contador;

    public HiloIncrementador(Contador c) {
        contador = c;
    }

    public void run() {
        for (int i = 0; i < 1000; i++) {
            contador.incrementar();
        }
    }
}

public class EjemploMonitor {
    public static void main(String[] args) {
        Contador contador = new Contador();

        // Crear dos hilos que comparten el mismo recurso
        HiloIncrementador t1 = new HiloIncrementador(contador);
        HiloIncrementador t2 = new HiloIncrementador(contador);

        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Valor final: " + contador.obtenerValor());
    }
}
```

- Contador: es el recurso compartido, protegido con métodos sincronizados.
- HiloIncrementador: extiende Thread y en su método run() incrementa el contador muchas veces.
- main: crea dos hilos y los ejecuta. Al final, se espera con join() a que terminen y se imprime el valor final.

De esta forma, el programa usa monitores (synchronized) pero sin recurrir a lambdas ni estilo funcional.

Condición de Carrera

- Una **condición de carrera** ocurre cuando dos o más hilos acceden y modifican un recurso compartido al mismo tiempo, sin la debida sincronización.
- El resultado final depende del orden en que se ejecuten las instrucciones de los hilos, lo cual es **no determinista**.
- En Java, esto suele pasar cuando varios hilos modifican una variable compartida sin usar synchronized.

```
class Contador {
    private int valor = 0;

    // Método sin sincronización
    public void incrementar() {
        valor++;
    }

    public int obtenerValor() {
        return valor;
    }
}

// Clase que representa un hilo que incrementa el contador
class HiloIncrementador extends Thread {
    private Contador contador;

    public HiloIncrementador(Contador c) {
        contador = c;
    }

    public void run() {
        for (int i = 0; i < 1000; i++) {
            contador.incrementar();
        }
    }
}

public class CondicionCarrera {
    public static void main(String[] args) {
        Contador contador = new Contador();

        // Crear dos hilos que comparten el mismo recurso
        HiloIncrementador t1 = new HiloIncrementador(contador);
        HiloIncrementador t2 = new HiloIncrementador(contador);

        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
```

```
        }

        // El valor esperado sería 2000, pero probablemente no lo sea
        System.out.println("Valor final: " + contador.obtenerValor());
    }
}
```

Qué sucede

- Cada hilo incrementa el contador 1000 veces.
- El valor esperado es **2000**, pero debido a la condición de carrera, el resultado puede ser menor (por ejemplo, 1987, 1995, etc.).
- Esto pasa porque la operación valor++ no es atómica: internamente se descompone en **leer, incrementar y escribir**, y los hilos pueden interrumpirse entre esos pasos.

Solución

- Para evitar la condición de carrera, se debe usar un **monitor** con synchronized:

De esta forma, solo un hilo puede ejecutar el método a la vez, y el resultado será siempre **2000**.

Deadlock

- Un **deadlock** ocurre cuando dos o más hilos quedan bloqueados esperando recursos que nunca se liberan.
- Cada hilo posee un recurso y espera otro que está en manos del otro hilo.
- Como ninguno libera lo que tiene, ambos quedan bloqueados indefinidamente.

```
class Recurso {
    private String nombre;

    public Recurso(String n) {
        nombre = n;
    }

    public String getNombre() {
        return nombre;
    }
}

class Tarea extends Thread {
```

```
    private Recurso recurso1;
    private Recurso recurso2;

    public Tarea(Recurso r1, Recurso r2) {
        recurso1 = r1;
        recurso2 = r2;
    }

    public void run() {
        synchronized (recurso1) {
            System.out.println(getName() + " bloqueó " + recurso1.getNombre());
            try {
                Thread.sleep(100); // simula trabajo
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

            System.out.println(getName() + " intenta bloquear " +
recurso2.getNombre());
            synchronized (recurso2) {
                System.out.println(getName() + " bloqueó " + recurso2.getNombre());
            }
        }
    }
}

public class EjemploDeadlock {
    public static void main(String[] args) {
        Recurso recursoA = new Recurso("Recurso A");
        Recurso recursoB = new Recurso("Recurso B");

        // Dos hilos que intentan bloquear recursos en orden inverso
        Tarea t1 = new Tarea(recursoA, recursoB);
        Tarea t2 = new Tarea(recursoB, recursoA);

        t1.setName("Hilo 1");
        t2.setName("Hilo 2");

        t1.start();
        t2.start();
    }
}
```

Qué ocurre

- **Hilo 1** bloquea Recurso A y luego intenta bloquear Recurso B.
- **Hilo 2** bloquea Recurso B y luego intenta bloquear Recurso A.
- Ninguno puede continuar porque cada uno espera el recurso que el otro ya tiene.
- Resultado: **deadlock**, ambos hilos quedan bloqueados para siempre.

Cómo evitarlo

- **Orden consistente**: siempre adquirir los recursos en el mismo orden.
- **Tiempo de espera**: usar bloqueos con timeout (tryLock en ReentrantLock).

- **Diseño cuidadoso**: minimizar la necesidad de múltiples bloqueos.

Livelock

- Un **livelock** ocurre cuando dos o más hilos **no están bloqueados** pero siguen **cambiando de estado** en respuesta a las acciones del otro, sin avanzar nunca en su trabajo.
- A diferencia del **deadlock** (donde los hilos quedan bloqueados esperando recursos), en el livelock los hilos están activos, pero atrapados en un ciclo de reacciones inútiles.
- Es como dos personas en un pasillo que intentan ceder el paso al mismo tiempo: se mueven constantemente, pero nadie avanza.

```
class Recurso {
    private String nombre;
    private boolean enUso = false;

    public Recurso(String n) {
        nombre = n;
    }

    public synchronized boolean intentarUsar() {
        if (!enUso) {
            enUso = true;
            return true;
        }
        return false;
    }

    public synchronized void liberar() {
        enUso = false;
    }

    public String getNombre() {
        return nombre;
    }
}

class Trabajador extends Thread {
    private String nombre;
    private Recurso recurso1;
    private Recurso recurso2;

    public Trabajador(String n, Recurso r1, Recurso r2) {
        nombre = n;
        recurso1 = r1;
```

```
        recurso2 = r2;
    }

    public void run() {
        while (true) {
            // Intenta usar el primer recurso
            if (recurso1.intentarUsar()) {
                System.out.println(nombre + " obtuvo " + recurso1.getNombre());

                // Si no puede obtener el segundo recurso, libera el primero
                if (!recurso2.intentarUsar()) {
                    System.out.println(nombre + " no pudo obtener " +
recurso2.getNombre() + ", libera " + recurso1.getNombre());
                    recurso1.liberar();
                    try {
                        Thread.sleep(100); // espera un poco antes de intentar de nuevo
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                } else {
                    // Si logra ambos recursos, termina
                    System.out.println(nombre + " obtuvo ambos recursos y terminó.");
                    recurso2.liberar();
                    recurso1.liberar();
                    break;
                }
            }
        }
    }
}

public class EjemploLivelock {
    public static void main(String[] args) {
        Recurso recursoA = new Recurso("Recurso A");
        Recurso recursoB = new Recurso("Recurso B");

        Trabajador t1 = new Trabajador("Trabajador 1", recursoA, recursoB);
        Trabajador t2 = new Trabajador("Trabajador 2", recursoB, recursoA);

        t1.start();
        t2.start();
    }
}
```

Qué ocurre

- Cada trabajador intenta obtener dos recursos.
- Si no logra el segundo, libera el primero y vuelve a intentar.
- Como ambos hilos hacen lo mismo, pueden entrar en un **livelock**: ninguno queda bloqueado, pero ambos siguen cediendo y retomando recursos sin progresar.
- El programa puede terminar si uno logra obtener ambos recursos, pero muchas veces se quedará en un ciclo interminable.

Conclusión

El **livelock** es más difícil de detectar que el deadlock porque los hilos parecen estar activos. La solución suele ser introducir **estrategias de espera aleatoria** o reglas claras de prioridad para evitar que los hilos cedan indefinidamente.

Mutex (Mutua exclusión)

- La **mutua exclusión** significa que **solo un hilo puede acceder a un recurso compartido en un momento dado**.
- En Java, se logra con la palabra clave synchronized, que convierte un método o bloque en una sección crítica.
- Esto evita **condiciones de carrera** y asegura que los datos compartidos se mantengan consistentes.

```
class Contador {
    private int valor = 0;

    // Método sincronizado: asegura mutua exclusión
    public synchronized void incrementar() {
        valor++;
    }

    public synchronized int obtenerValor() {
        return valor;
    }
}

// Clase que representa un hilo que incrementa el contador
class HiloIncrementador extends Thread {
    private Contador contador;

    public HiloIncrementador(Contador c) {
        contador = c;
    }

    public void run() {
        for (int i = 0; i < 1000; i++) {
            contador.incrementar();
        }
    }
}

public class EjemploMutuaExclusion {
    public static void main(String[] args) {
        Contador contador = new Contador();

        // Crear dos hilos que comparten el mismo recurso
        HiloIncrementador t1 = new HiloIncrementador(contador);
        HiloIncrementador t2 = new HiloIncrementador(contador);

        t1.start();
        t2.start();
```

```
        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        // Gracias a la mutua exclusión, el resultado será siempre correcto
        System.out.println("Valor final: " + contador.obtenerValor());
    }
}
```

Qué ocurre

- **Sin** synchronized: los hilos podrían pisarse entre sí y el resultado sería menor a 2000 (condición de carrera).
- **Con** synchronized: se garantiza que solo un hilo entra al método incrementar() a la vez, logrando **mutua exclusión**.
- Resultado final: **2000**, siempre correcto.

Conclusión

La **mutua exclusión** es la base de la programación concurrente segura en Java. Se implementa fácilmente con synchronized, y es fundamental para evitar inconsistencias en recursos compartidos.

¿Quieres que te muestre también un ejemplo usando **bloques sincronizados** (synchronized(this)) en lugar de métodos sincronizados, para ver otra forma de aplicar mutua exclusión?

Happens-before

- El modelo de memoria de Java define reglas de **visibilidad y orden de ejecución** entre hilos.
- La relación **happens-before** asegura que las operaciones realizadas por un hilo sean **visibles y ordenadas** para otro hilo.
- Ejemplos de relaciones happens-before:
    - Una llamada a unlock() en un Lock ocurre antes de un posterior lock() en el mismo objeto.
    - Una escritura en una variable volatile ocurre antes de una lectura posterior de esa misma variable.
    - La finalización de un hilo (Thread.join()) ocurre antes de que el hilo que llamó a join() continúe.

En otras palabras: si una acción A **happens-before** una acción B, entonces los efectos de A son visibles para B.

```java
class DatosCompartidos {
    private int valor = 0;

    public synchronized void escribir(int nuevoValor) {
        valor = nuevoValor;
        System.out.println("Escrito: " + nuevoValor);
    }

    public synchronized int leer() {
        System.out.println("Leído: " + valor);
        return valor;
    }
}

class Escritor extends Thread {
    private DatosCompartidos datos;

    public Escritor(DatosCompartidos d) {
        datos = d;
    }

    public void run() {
        datos.escribir(42);
    }
}

class Lector extends Thread {
    private DatosCompartidos datos;

    public Lector(DatosCompartidos d) {
        datos = d;
    }

    public void run() {
        datos.leer();
    }
}

public class EjemploHappensBefore {
    public static void main(String[] args) {
        DatosCompartidos datos = new DatosCompartidos();

        Escritor escritor = new Escritor(datos);
        Lector lector = new Lector(datos);

        escritor.start();
        try {
            escritor.join(); // asegura que la escritura happens-before la lectura
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        lector.start();
    }
}
```

Qué ocurre

- El método escribir() es **synchronized**, lo que garantiza exclusión mutua y visibilidad.
- El join() asegura que el hilo escritor termine antes de que el lector empiece.
- Por la relación **happens-before**, el valor escrito (42) será visible para el lector.

Conclusión

El concepto de **happens-before** es fundamental en la concurrencia de Java:

- Define el orden en que las operaciones de distintos hilos se ven entre sí.
- Evita resultados inconsistentes o invisibles.
- Se logra mediante mecanismos como synchronized, volatile, join(), y las clases de java.util.concurrent.

¿Quieres que te muestre también un ejemplo usando una **variable** volatile para ilustrar happens-before sin necesidad de synchronized?

Bloqueo Intrínseco

- En Java, cada objeto tiene asociado un **bloqueo intrínseco** (también llamado *monitor lock*).
- Cuando un método o bloque está marcado con synchronized, el hilo que lo ejecuta debe adquirir el bloqueo intrínseco del objeto antes de entrar.
- Mientras un hilo posee ese bloqueo, ningún otro hilo puede ejecutar métodos sincronizados sobre el mismo objeto.
- Esto garantiza **mutua exclusión** y orden de acceso a recursos compartidos.

```
class Contador {
    private int valor = 0;

    // Método sincronizado: requiere el bloqueo intrínseco del objeto Contador
    public synchronized void incrementar() {
        valor++;
        System.out.println(Thread.currentThread().getName() + " incrementó a " +
valor);
    }

    public synchronized int obtenerValor() {
        return valor;
    }
}

class HiloIncrementador extends Thread {
    private Contador contador;

    public HiloIncrementador(Contador c, String nombre) {
        super(nombre);
        contador = c;
    }

    public void run() {
        for (int i = 0; i < 5; i++) {
            contador.incrementar();
            try {
                Thread.sleep(100); // simula trabajo
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public class EjemploBloqueoIntrinseco {
    public static void main(String[] args) {
        Contador contador = new Contador();

        HiloIncrementador t1 = new HiloIncrementador(contador, "Hilo 1");
        HiloIncrementador t2 = new HiloIncrementador(contador, "Hilo 2");

        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Valor final: " + contador.obtenerValor());
    }
```

```
}
```

Qué ocurre

- Cada vez que un hilo entra en incrementar() o obtenerValor(), adquiere el **bloqueo intrínseco** del objeto contador.
- Mientras un hilo posee el bloqueo, el otro debe esperar.
- Esto asegura que el valor se incremente correctamente sin condiciones de carrera.

Conclusión

El **bloqueo intrínseco** es el mecanismo básico de sincronización en Java:

- Se activa automáticamente con synchronized.
- Garantiza que solo un hilo acceda a la sección crítica a la vez.
- Es fundamental para implementar monitores y asegurar la consistencia de los datos compartidos.

Starvation

- La **starvation** ocurre cuando un hilo nunca logra acceder a un recurso compartido porque otros hilos de mayor prioridad o con más oportunidades lo acaparan.
- El hilo afectado sigue vivo, pero **no progresa** porque siempre queda esperando.
- Esto puede suceder, por ejemplo, cuando se usan prioridades de hilos o cuando un sistema de bloqueo favorece constantemente a ciertos hilos.

```
class RecursoCompartido {
    public synchronized void usar(String nombreHilo) {
        System.out.println(nombreHilo + " está usando el recurso.");
        try {
            Thread.sleep(500); // simula trabajo
        } catch (InterruptedException e) {
```

```
            e.printStackTrace();
        }
        System.out.println(nombreHilo + " terminó de usar el recurso.");
    }
}

class HiloTrabajador extends Thread {
    private RecursoCompartido recurso;

    public HiloTrabajador(RecursoCompartido r, String nombre) {
        super(nombre);
        recurso = r;
    }

    public void run() {
        while (true) {
            recurso.usar(getName());
            try {
                Thread.sleep(100); // espera antes de intentar de nuevo
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public class EjemploStarvation {
    public static void main(String[] args) {
        RecursoCompartido recurso = new RecursoCompartido();

        // Crear varios hilos con distintas prioridades
        HiloTrabajador hilo1 = new HiloTrabajador(recurso, "Hilo 1 (Alta prioridad)");
        HiloTrabajador hilo2 = new HiloTrabajador(recurso, "Hilo 2 (Media prioridad)");
        HiloTrabajador hilo3 = new HiloTrabajador(recurso, "Hilo 3 (Baja prioridad)");

        hilo1.setPriority(Thread.MAX_PRIORITY); // prioridad alta
        hilo2.setPriority(Thread.NORM_PRIORITY); // prioridad normal
        hilo3.setPriority(Thread.MIN_PRIORITY); // prioridad baja

        hilo1.start();
        hilo2.start();
        hilo3.start();
    }
}
```

Qué ocurre

- El **Hilo 1** tiene prioridad máxima, por lo que el planificador de hilos de la JVM tiende a darle más tiempo de CPU.
- El **Hilo 3**, con prioridad mínima, puede quedar **hambriento (starvation)** porque rara vez se le asigna tiempo de ejecución.
- Aunque el hilo está vivo, apenas progresa porque los otros hilos lo desplazan constantemente.

Conclusión

La **starvation** es un problema de equidad en la concurrencia:

- No se trata de bloqueo total (como el deadlock), sino de falta de oportunidades.
- Se puede mitigar usando políticas de planificación más justas, como **fair locks** (ReentrantLock con opción de equidad) o evitando el uso excesivo de prioridades.

Thread-safe

- **Thread-safe** significa que un código puede ser ejecutado por múltiples hilos al mismo tiempo sin producir resultados incorrectos ni condiciones de carrera.
- En Java, se logra mediante:
    - **Bloqueo intrínseco** con synchronized.
    - Variables volatile para asegurar visibilidad.
    - Clases de la librería java.util.concurrent (como AtomicInteger, ConcurrentHashMap, etc.).
- La idea es que los datos compartidos estén protegidos y siempre se mantenga la consistencia.

```
class ContadorSeguro {
    private int valor = 0;

    // Métodos sincronizados: garantizan thread-safety
    public synchronized void incrementar() {
        valor++;
    }

    public synchronized int obtenerValor() {
        return valor;
    }
}

class HiloIncrementador extends Thread {
    private ContadorSeguro contador;

    public HiloIncrementador(ContadorSeguro c, String nombre) {
        super(nombre);
        contador = c;
    }

    public void run() {
        for (int i = 0; i < 1000; i++) {
```

```
            contador.incrementar();
        }
        System.out.println(getName() + " terminó.");
    }
}

public class EjemploThreadSafe {
    public static void main(String[] args) {
        ContadorSeguro contador = new ContadorSeguro();

        HiloIncrementador t1 = new HiloIncrementador(contador, "Hilo 1");
        HiloIncrementador t2 = new HiloIncrementador(contador, "Hilo 2");

        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        // Siempre será 2000 gracias a la seguridad de hilos
        System.out.println("Valor final: " + contador.obtenerValor());
    }
}
```

Qué ocurre

- Dos hilos incrementan el mismo contador 1000 veces cada uno.
- Sin sincronización, el resultado podría ser menor a 2000 (condición de carrera).
- Con synchronized, el código es **thread-safe**: el valor final siempre será correcto.

Conclusión

- **Thread-safe** significa que el código funciona correctamente en entornos concurrentes.
- En Java, se logra con synchronized, volatile o estructuras de concurrencia avanzadas.
- Es esencial cuando varios hilos comparten y modifican datos.

Semaphore

Un Semaphore es un mecanismo de sincronización que controla el acceso a uno o más recursos mediante un contador. A diferencia del mutex (que permite solo 1 acceso), el semáforo permite n accesos simultáneos.

Analogía: Un estacionamiento con 3 espacios disponibles:

- Cada auto que entra adquiere un permiso (espacio).
- Cada auto que sale libera un permiso.
- Si no hay espacios, los autos esperan.

```
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;

public class EjemploBasicoSemaphore {
    public static void main(String[] args) {
        // Semáforo con 3 permisos (permite 3 hilos concurrentes)
        Semaphore parking = new Semaphore(3);

        // Crear 10 hilos (autos) que intentan estacionar
        for (int i = 1; i <= 10; i++) {
            new Thread(new Auto(parking, "Auto-" + i)).start();
        }
    }
}

class Auto implements Runnable {
    private final Semaphore parking;
    private final String nombre;

    public Auto(Semaphore parking, String nombre) {
        this.parking = parking;
        this.nombre = nombre;
    }

    @Override
    public void run() {
        try {
            System.out.println(nombre + " quiere estacionar...");

            // Adquirir un permiso (bloquea si no hay disponibles)
            parking.acquire();

            System.out.println(nombre + " ESTACIONÓ. Espacios restantes: "
                               + parking.availablePermits());

            // Simular tiempo estacionado
            Thread.sleep(2000);
```

```
            System.out.println(nombre + " se va. Espacios restantes: "
                               + (parking.availablePermits() + 1));

            // Liberar el permiso
            parking.release();

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Liveness

Liveness (capacidad de "seguir vivo") es la propiedad de un sistema concurrente que garantiza que los hilos **siempre progresan** hacia su objetivo, sin quedarse atascados indefinidamente.

Los problemas de falta de liveness son:

- **Deadlock** - Hilos bloqueados permanentemente.
- **Starvation** - Hilo nunca obtiene recursos.
- **Livelock** - Hilos cambian estado, pero no progresan.